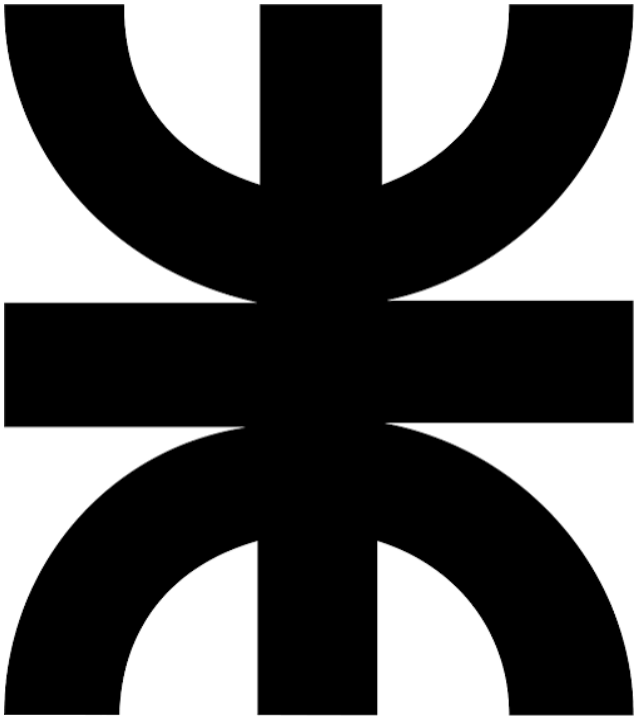


UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA

-Desarrollo de Software-

Temas de Investigación: BDD con Gherkin y Mutation Testing

Grupo N°13

Integrantes:

- Barberán Aquino, Facundo
- Bassi, Aymará Aylén
- Escalante, Alejo Leonel
- Franco, Ivan
- Gomez Samardich, Agustín Nicolás
- Lopez, Agustin
- Mendez Nicolet, Gustavo Joaquin
- Quiroz, Luis Ariel
- Servín, Santiago Nicolás

Ciclo lectivo 2026

Índice

Contenido

Índice 2
1. Introducción 4
2. Investigación conceptual: BDD con Gherkin 5
2.1 Qué es BDD 5
2.2 Qué problema resuelve 5
2.3 Características principales 5
2.4 Sintaxis de Gherkin 5
2.5 Proceso de trabajo 6
2.6 Herramientas 6
2.7 Resultados posibles 7
2.8 Beneficios y limitaciones 7
3. Investigación conceptual: Mutation Testing 8
3.1 Qué es Mutation Testing 8
3.2 Qué problema resuelve 8
3.3 Conceptos clave y operadores de mutación 8
3.4 Estados de un mutante y puntuación 9
3.5 Proceso de trabajo 10
3.6 Herramientas 11
3.7 Beneficios y limitaciones 11
4. Relación con el Trabajo Práctico Integrador 11
4.1 Aplicación de BDD con Gherkin en M5 11
4.2 Aplicación de Mutation Testing en M5 12
4.4 Uso combinado y alcance 13
5. Criterios de utilización y análisis crítico 14
5.1 BDD con Gherkin: cuándo aplicarlo y cuándo no 14
5.2 Mutation Testing: cuándo aplicarlo y cuándo no 14
5.3 Costos, rendimiento y mantenibilidad 15
5.4 Alternativas y complementos 15
5.5 Decisiones técnicas adoptadas 16
6. Implementación práctica integrada 16

6.1 Objetivo y escenario elegido 16
6.2 Estructura del proyecto 17
6.3 Código, escenarios y step definitions 17
6.4 Contenedor Docker 19
6.5 Ejecución y evidencias 19
6.6 Aporte de la integración 24
7. Conclusiones 24
8. Repositorio de código 25
9. Fuentes 25

1. Introducción

Este informe presenta la investigación del Grupo 13 sobre dos técnicas que abordan problemas frecuentes en el desarrollo de software: por un lado, que los requerimientos se interpreten de forma distinta entre quienes los definen y quienes los implementan; por otro, que las pruebas automatizadas den una falsa sensación de seguridad, porque pasan en verde aunque no verifiquen todos los comportamientos importantes. Las técnicas investigadas son BDD con Gherkin y Mutation Testing. BDD (*Behavior Driven Development*) propone definir el comportamiento del sistema con ejemplos concretos, escritos en un lenguaje que entienden tanto las personas técnicas como las no técnicas, y convertir esos ejemplos en pruebas ejecutables. Mutation Testing, en cambio, evalúa la calidad de las pruebas existentes: introduce errores pequeños y deliberados en el código y comprueba si las pruebas son capaces de detectarlos.

Ambas técnicas se complementan. BDD responde a la pregunta *qué debe hacer el sistema*, mientras que Mutation Testing responde a *si las pruebas realmente lo verifican*. Usadas en conjunto, permiten especificar el comportamiento esperado, medir qué tan sólidas son las pruebas que lo controlan y completar los casos que falten.

A lo largo del trabajo se desarrollan los fundamentos de cada técnica, sus características, beneficios y limitaciones. También se analiza cómo podrían aplicarse en la plataforma distribuida de movilidad urbana desarrollada en el Trabajo Práctico Integrador, en particular en el módulo M5 (Solicitud y Despacho), y en qué situaciones conviene utilizarlas o no, considerando su complejidad, costo, rendimiento y mantenibilidad. Por último, se presenta una implementación práctica que integra ambas técnicas en un mismo escenario, desarrollada en Python con las herramientas pytest-bdd y mutmut, ejecutada dentro de un contenedor Docker y publicada en un repositorio Git.

2. Investigación conceptual: BDD con Gherkin

2.1 Qué es BDD

BDD (*Behavior Driven Development*, desarrollo guiado por comportamiento) es una metodología derivada de TDD (*Test-Driven Development*) que define el comportamiento del sistema desde la perspectiva del usuario antes de escribir el código. Fue propuesta por Dan North a partir de 2006 para responder a dudas habituales de TDD: por dónde empezar, qué probar y cómo nombrar las pruebas.

Gherkin es el DSL (*Domain-Specific Language*) legible por humanos que formaliza esos comportamientos mediante la estructura Given / When / Then (Dado / Cuando / Entonces). Permite escribir especificaciones que entiende cualquier integrante del equipo y que, al mismo tiempo, se ejecutan como pruebas automatizadas.

2.2 Qué problema resuelve

BDD resuelve la brecha de comunicación entre quienes definen los requerimientos (Product Owner, analistas) y quienes los implementan y verifican (desarrolladores, QA). Obliga a discutir los criterios de aceptación con ejemplos concretos antes de programar, lo que reduce el retrabajo causado por especificaciones ambiguas o interpretaciones distintas de un mismo requerimiento.

2.3 Características principales

- **Archivos de especificación independientes** con extensión `.feature`, separados del código de la aplicación.
- **Documentación viva** (*living documentation*): los escenarios se ejecutan contra el código, por lo que fallan si el sistema se desvía de lo documentado.
- **Mapeo de lenguaje natural a código** mediante *step definitions*: funciones que conectan cada frase del escenario con una llamada al código de producción y una aserción.
- **Estilo declarativo:** los escenarios describen reglas de negocio (qué debe pasar), no detalles de la interfaz (dónde hacer clic).

2.4 Sintaxis de Gherkin

Palabra clave (español / inglés)	Función
Característica / Feature	Describe a alto nivel una funcionalidad del software y agrupa escenarios relacionados.
Escenario / Scenario	Ejemplo concreto que ilustra una regla de negocio.
Dado / Given	Contexto inicial: pone al sistema en un estado concreto antes de la acción.
Cuando / When	Evento o acción, realizada por una persona o por otro sistema.
Entonces / Then	Resultado esperado; se verifica con una aserción que compara lo real con lo esperado.

Palabra clave (español / inglés)	Función
Y / Pero (And / But)	Encadenan pasos adicionales del mismo tipo.
Antecedentes / Background	Pasos comunes que se repiten antes de cada escenario de la característica.
Esquema del escenario + Ejemplos / Scenario Outline + Examples	Un mismo escenario ejecutado con varias filas de datos de una tabla.
Etiquetas (`@limite`)	Clasifican escenarios para filtrarlos o seleccionarlos al ejecutar.

Ejemplo tomado de la demo:

```
# language: es
Característica: Retiro de dinero en Cajero Automatico

  Escenario: Retiro exitoso cuando hay saldo suficiente
    Dado que el cliente tiene un saldo disponible de 1000 pesos
    Cuando solicita retirar 300 pesos
    Entonces la operacion es "RETIRO_EXITOSO"
    Y el nuevo saldo disponible es de 700 pesos
```

2.5 Proceso de trabajo

El proceso es iterativo y se centra en la colaboración antes de la implementación:

1. **Conversar con ejemplos:** negocio, desarrollo y QA acuerdan casos concretos y comportamientos esperados, incluidos los casos de borde.
2. **Redactar los escenarios** en un archivo `.feature` con Gherkin, en estilo declarativo orientado al negocio.
3. **Ejecutar (fallo inicial):** los pasos sin implementar o sin código real deben fallar.
4. **Escribir las step definitions:** código (por ejemplo con pytest-bdd) que traduce cada paso a una llamada al código de producción y a un `assert`.
5. **Implementar y refactorizar** hasta que todos los escenarios pasen; luego sirven de respaldo para refactorizaciones futuras.
6. **Mantenimiento continuo:** si una regla de negocio cambia, el escenario falla y alerta al equipo.

Un requisito importante es que las step definitions **invoquen el código real** de la aplicación. Si el paso simula el resultado dentro del test, el escenario pasa siempre y no verifica nada.

2.6 Herramientas

Herramienta	Ecosistema	Característica
pytest-bdd	Python	Plugin que integra BDD dentro de pytest; aprovecha fixtures, marcadores y configuración del repositorio. Depende de gherkin-official, parse, parse-type y Mako.

Herramienta	Ecosistema	Característica
behave	Python	Alternativa con ejecutor propio y estructura de directorios propia (`features/` y `steps/`).
Cucumber	Java, JavaScript, Ruby y otros	Herramienta de referencia global para la sintaxis Gherkin.

2.7 Resultados posibles

- **Aprobado (Passed):** el comportamiento del sistema coincide con lo esperado.
- **Fallido (Failed):** una aserción falló porque el comportamiento real difiere del escenario acordado.
- **Paso sin definición (StepDefinitionNotFoundError):** falta la función de Python asociada a una frase del Gherkin.
- **Error por excepción:** ocurrió un error no capturado durante un paso.
- **No seleccionado (Deselected):** el escenario fue excluido mediante etiquetas.

Las métricas de BDD son el conteo de escenarios aprobados y fallidos por característica, junto con la documentación viva generada.

2.8 Beneficios y limitaciones

Beneficios	Limitaciones
Lenguaje compartido: sintaxis legible que reduce la brecha entre perfiles técnicos y no técnicos.	**Costo de mantenimiento:** exige coordinar tres capas (escenarios, step definitions y código).
Documentación viva: los `.feature` fallan si el sistema se desvía de lo documentado.	**Requiere disciplina:** sin participación del negocio queda como una capa extra sobre pruebas de integración.
Detección temprana de malentendidos: los criterios se discuten antes de programar.	**Sobre especificación imperativa:** describir clics o campos vuelve las pruebas frágiles.
Foco en el valor de negocio: evita implementar funciones innecesarias.	**Velocidad reducida:** los escenarios de integración o extremo a extremo tardan más que las pruebas unitarias.
Reutilización de pasos: una step definition sirve para muchos escenarios.	**Gestión de datos de prueba:** preparar el estado inicial (Dado) exige fixtures, semillas de base de datos o dobles.
	Alcance limitado: no reemplaza pruebas de rendimiento, concurrencia, seguridad o contratos.

3. Investigación conceptual: Mutation Testing

3.1 Qué es Mutation Testing

Mutation Testing (pruebas de mutación) es una técnica que evalúa la calidad de las pruebas existentes introduciendo cambios pequeños y deliberados en el código que se está probando. Cada versión modificada se denomina mutante y representa un posible error. Luego se ejecutan las pruebas contra cada mutante: si al menos una prueba falla, significa que las pruebas detectaron el error, y se dice que el mutante fue **eliminado** (*killed*); el mutante no se borra, sino que queda marcado como detectado. Si todas las pruebas siguen pasando a pesar del cambio, el error pasó desapercibido y se dice que el mutante **sobrevivió**, lo que indica que falta un caso o una verificación en la suite.

La pregunta que responde es concreta: *si introdujéramos este error, ¿las pruebas lo detectarían?*

3.2 Qué problema resuelve

Una prueba puede ejecutar una función y no verificar su salida. La cobertura de líneas indica qué instrucciones ejecutan las pruebas, y la cobertura de ramas, qué alternativas de decisión recorren. Esas mediciones no demuestran, por sí solas, que se comprueben los resultados correctos.

Mutation Testing complementa esas métricas al observar si las pruebas distinguen el comportamiento original de versiones alteradas. Por ejemplo, las condiciones `monto <= 0` y `monto < 0` se comportan igual para cualquier monto distinto de cero; solo difieren en el límite exacto. Si ninguna prueba usa el valor cero, el error pasa desapercibido aunque la cobertura sea del 100 %.

3.3 Conceptos clave y operadores de mutación

Un operador de mutación define la transformación que se aplica al código. Los operadores disponibles dependen de la herramienta y del lenguaje.

Operador	Ejemplo de cambio
Límite relacional	`>` → `>=`, `<=` → `<`
Constante numérica	`0` → `1`
Operación aritmética o de asignación	`-=` → `+=`, `-=` → `=`
Negación de condición	`if x` → `if not x`
Alteración de retorno o de cadena	`"MONTO_INVALIDO"` → `"XXMONTO_INVALIDOXX"`
Reemplazo por nulo	`self.saldo = saldo` → `self.saldo = None`

Las mutaciones se generan solo en el entorno de pruebas; nunca se introducen en el sistema desplegado.

3.4 Estados de un mutante y puntuación

Estado	Significado	¿Es bueno o malo?
Eliminado (Killed)	Al menos una prueba falló con el mutante. Es decir, las pruebas detectaron el error introducido. El mutante no se borra: queda registrado como detectado.	Bueno.
Sobreviviente (Survived)	Todas las pruebas siguieron pasando, aunque el código estaba modificado. El error pasó desapercibido.	Malo: falta un caso o una verificación.
Sin cobertura (No coverage)	El mutante está en una parte del código que ninguna prueba ejecuta.	Malo: esa parte no se prueba.
Tiempo de espera / Error (Timeout / Error)	El cambio hizo que el programa no terminara, por ejemplo, porque entró en un bucle infinito. La herramienta corta la ejecución después de un tiempo límite.	Hay que revisarlo.

La interpretación depende de la herramienta: Cada herramienta de mutación decide cómo clasificar algunos de estos casos. Por ejemplo, Stryker Mutator (una herramienta de mutation testing para JavaScript, TypeScript y C#) cuenta los *timeouts* como mutantes detectados: si el cambio colgó el programa, considera que el error "se notó". Otras herramientas los informan por separado. Por eso, el mismo código puede dar porcentajes distintos según la herramienta utilizada.

Puntuación de mutación: La métrica principal de la técnica es la puntuación de mutación (*mutation score*). Indica qué porcentaje de los mutantes evaluados fue detectado por las pruebas:

$$Puntuacion = \frac{eliminados}{eliminados + sobrevivientes} \times 100$$

Mutantes equivalentes: Algunos mutantes cambian el código, pero no cambian lo que el programa hace. Por ejemplo, si x es un número entero, las condiciones x > 0 y x >= 1 son exactamente iguales: ninguna prueba puede distinguirlas. Estos mutantes se llaman **equivalentes**, y es imposible eliminarlos. Por eso, la bibliografía recomienda no contarlos en la fórmula una vez identificados, para no bajar la puntuación injustamente. Por ejemplo, con 9 eliminados, 1 sobreviviente y 2 equivalentes, la puntuación se calcula como 9 / (9 + 1) = 90 %, y no como 9 / 12 = 75 %.

Cómo informar el resultado: El porcentaje solo no alcanza, no es lo mismo un 100 % sobre 3 mutantes que sobre 500. Por eso hay que informar siempre la cantidad de mutantes evaluados, cuántos fueron eliminados y cuántos sobrevivieron, y si se excluyó alguno (por ejemplo, por ser equivalente).

Además, una puntuación del 100 % significa que las pruebas detectaron todos los mutantes que generó la herramienta. No significa que el programa no tenga errores, ni que todos los requisitos estén cubiertos. Por ejemplo, si un sistema debe respetar una regla que nunca se programó ni se especificó en una prueba, ninguna herramienta de mutación la va a evaluar: la puntuación puede ser del 100 % y la regla seguir faltando.

3.5 Proceso de trabajo

1. **Establecer una línea base:** Antes de introducir mutaciones, todas las pruebas deben pasar sobre el código original. Esta condición es necesaria para poder interpretar los resultados: si una prueba ya falla sin cambios, no se puede saber si un fallo posterior lo provocó el mutante o un problema que ya existía. Si la suite falla o es inestable, primero hay que corregirla.
2. **Selección y generación:** Se define qué parte del código se va a evaluar. Normalmente se elige el código de producción que contiene la lógica de negocio, y se excluyen las pruebas, el código generado automáticamente y las dependencias externas. Sobre ese código se aplican los operadores de mutación y se generan los mutantes. Cada mutante contiene, por lo general, un solo cambio: así, si sobrevive, se sabe con exactitud qué modificación no fue detectada.
3. **Ejecución contra mutantes:** Se define qué parte del código se va a evaluar. Normalmente se elige el código de producción que contiene la lógica de negocio, y se excluyen las pruebas, el código generado automáticamente y las dependencias externas. Sobre ese código se aplican los operadores de mutación y se generan los mutantes. Cada mutante contiene, por lo general, un solo cambio: así, si sobrevive, se sabe con exactitud qué modificación no fue detectada.
4. **Inspección de sobrevivientes:** Cada mutante que sobrevive se analiza para entender por qué las pruebas no lo detectaron. Las causas posibles son:
    - Falta una prueba o una verificación: ningún caso ejercita esa situación, o el caso existe pero no comprueba el resultado. Es la causa más frecuente y la que la técnica busca encontrar.
    - Mutante equivalente: el cambio no altera el comportamiento del programa, por lo que ninguna prueba puede detectarlo. Se documenta y se excluye del cálculo de la puntuación.
    - Código innecesario: si una parte del código puede modificarse sin que el comportamiento cambie, es posible que esa parte sobre.
    - Problema de ejecución: pruebas inestables o que dependen de factores externos, como servicios remotos o datos compartidos.
5. **Mejora y reejecución:** Por cada sobreviviente que indica una prueba faltante, se agrega o corrige una prueba. Esa prueba debe escribirse a partir de los requisitos: tiene que describir el comportamiento esperado del sistema, no copiar la lógica del código solo para eliminar el mutante. Luego se vuelve a ejecutar el análisis para confirmar la mejora. El ciclo se repite hasta que los sobrevivientes restantes sean equivalentes o hasta que el equipo decida, de forma justificada, no cubrirlos.

3.6 Herramientas

Herramienta	Ecosistema	Característica
mutmut	Python	Se ejecuta sobre pytest alterando el código fuente. Requiere la llamada al sistema `fork()`, por lo que en Windows debe ejecutarse en WSL o en un contenedor Linux.
PIT (PITest)	Java / JVM	Herramienta estándar de mutación para el ecosistema Java.
Stryker Mutator	JavaScript, TypeScript, C#, Scala	Framework de mutación con reportes de estados y métricas.
coverage.py	Python	Complementaria: mide cobertura de líneas y ramas antes o en paralelo a la mutación.

3.7 Beneficios y limitaciones

Beneficios	Limitaciones
Evaluación estricta de la suite: localiza casos de borde ausentes y aserciones débiles.	**Alto costo de procesamiento:** la suite se ejecuta una vez por mutante, cientos o miles de veces.
Evidencia para mejoras: indica con precisión qué verificaciones faltan, antes de refactorizar.	**Costo humano:** revisar sobrevivientes uno por uno lleva tiempo.
Métrica más exigente que la cobertura: mide si las pruebas verifican resultados, no solo si ejecutan líneas.	**Mutantes equivalentes:** generan falsas alarmas.
Red de seguridad ante regresiones en reglas críticas.	**Dependencia de la herramienta:** los resultados dependen de los operadores disponibles.
	Garantía limitada: si una regla nunca se especificó, las pruebas la ignoran aunque eliminen todos los mutantes.
	Alcance limitado: no evalúa rendimiento, seguridad, contratos ni concurrencia.

4. Relación con el Trabajo Práctico Integrador

4.1 Aplicación de BDD con Gherkin en M5

BDD se aplica a las reglas de negocio que el equipo debe acordar con claridad, en especial las que tienen límites temporales o de estado:

Proceso de M5	Regla a especificar en Gherkin
Creación y vencimiento de ofertas (RF-5.4)	La oferta se crea en estado PENDIENTE y vence exactamente al cumplirse su plazo.
Respuesta a una oferta	Solo el conductor destinatario puede aceptarla y solo mientras está vigente.
Asignación única	Si dos conductores aceptan, una sola asignación resulta ganadora.
Creación de solicitudes	Una cuenta no habilitada por M2 no puede crear una solicitud.
Estimación de precio	Consultar un precio no crea una solicitud ni reserva un conductor.

Ejemplo de característica para RF-5.4:

```
# language: es
Característica: Expiración de ofertas de viaje (RF-5.4)
  Como conductor disponible
  Quiero recibir ofertas con un tiempo límite de respuesta
  Para que el viaje no quede bloqueado indefinidamente

  Escenario: Creación exitosa de una oferta con tiempo de expiración
    Dado que existe una solicitud de viaje en estado "PENDIENTE"
    Cuando se genera una oferta con un tiempo de expiración de 30 segundos
    Entonces la oferta se crea en estado "PENDIENTE"
    Y la fecha de expiración es 30 segundos posterior al envío

  @limite
  Escenario: La oferta vence en el instante exacto del plazo
    Dado una oferta enviada con un plazo de 30 segundos
    Cuando pasaron 30 segundos desde el envío
    Entonces la oferta está "VENCIDA"
```

4.2 Aplicación de Mutation Testing en M5

El módulo M5 concentra reglas de negocio cuyo error afecta directamente a un viaje: aceptar una oferta vencida, permitir que responda un conductor que no es el destinatario o asignar dos conductores a la misma solicitud. Son reglas pequeñas, con resultados fáciles de observar y que se ejecutan rápido. Esa combinación las convierte en buenas candidatas para Mutation Testing, porque el costo del análisis es bajo y cada mutante sobreviviente es fácil de interpretar.

Procesos donde aplicarlo

- **Aceptación de ofertas**: La regla establece que solo el conductor destinatario puede aceptar una oferta, que la oferta debe estar vigente y que la solicitud no debe haber sido asignada antes. Una mutación relevante es cambiar la condición de vencimiento ahora >= vence_en por ahora > vence_en. Para detectarla hace falta una prueba en el instante exacto del vencimiento. Otra

mutación posible es invertir la verificación del destinatario. Para detectarla hace falta un caso en el que responde un conductor distinto, y la oferta tiene que ser rechazada.

- **Creación de solicitudes**: depende de la validación de la cuenta en M2. Si la herramienta invierte la condición de validación, las pruebas solo lo van a detectar si incluyen tanto una cuenta habilitada, con la que la solicitud se crea, como una cuenta no habilitada, con la que se rechaza. Si las pruebas solo prueban uno de los dos casos, el mutante sobrevive.

Ejemplo: regla de aceptación de ofertas

```
def evaluar_aceptacion(es_destinatario, ahora, vence_en, ya_asignada):
    if not es_destinatario:
        return "NO_AUTORIZADO"
    if ahora >= vence_en:
        return "VENCIDA"
    if ya_asignada:
        return "YA_ASIGNADA"
    return "ACEPTABLE"
```

Supongamos que la oferta vence en el instante 100 y que las pruebas usan cuatro casos: una aceptación válida en el instante 99, que debe dar ACEPTABLE; una respuesta de otro conductor en el instante 99, que debe dar NO_AUTORIZADO; una respuesta tardía en el instante 101, que debe dar VENCIDA; y una respuesta a una solicitud ya asignada, que debe dar YA_ASIGNADA.

Estos cuatro casos recorren todas las ramas de la función, así que la cobertura es del 100 %. Sin embargo, si la herramienta cambia ahora >= vence_en por ahora > vence_en, el mutante sobrevive, porque en los instantes 99 y 101 ambas condiciones dan el mismo resultado. La única diferencia está en el instante 100 exacto, y ninguna prueba lo usa. Para eliminar el mutante hay que agregar un quinto caso: una respuesta del destinatario en el instante 100, que debe dar VENCIDA.

El ejemplo también deja a la vista una decisión de negocio. El contrato exige que la oferta esté vigente, pero no aclara si en el instante exacto del plazo todavía lo está. El mutante sobreviviente obliga al equipo a acordar esa regla de forma explícita y a dejarla escrita en una prueba.

4.4 Uso combinado y alcance

En M5 ambas técnicas se complementan: BDD documenta y verifica las reglas acordadas (vencimiento, destinatario, asignación única), y Mutation Testing evalúa si esos escenarios detectan errores en los límites. Cuando un mutante sobrevive, la corrección se expresa como un nuevo escenario en Gherkin, que queda como documentación viva para todo el equipo.

Alcance y límites:

- La comprobación previa de `ya_asignada` no garantiza exclusión ante dos respuestas simultáneas. La asignación definitiva requiere una operación atómica en la base de datos (actualización condicional dentro de una transacción) y pruebas específicas de concurrencia.
- Valores como `es_destinatario` deben derivarse de la identidad autenticada y de la oferta persistida, no aceptarse como datos enviados por el cliente.
- La mutación conviene limitarla a la capa de dominio de M5; las integraciones con otros módulos se validan con pruebas de contrato.

5. Criterios de utilización y análisis crítico

5.1 BDD con Gherkin: cuándo aplicarlo y cuándo no

Conviene aplicarlo cuando:

- El sistema tiene lógica o reglas de negocio complejas.
- Los requerimientos tienen ambigüedades en sus límites temporales o de estado, y hay que acordar las reglas antes de programar.
- El equipo es multidisciplinario (PO, QA, desarrolladores) y se busca eliminar la brecha de comunicación.
- Existen flujos de usuario críticos (por ejemplo, aceptación o rechazo de un viaje).
- Se necesita documentación viva que se mantenga actualizada automáticamente con la suite de pruebas.

Conviene limitarlo cuando:

- El proyecto es simple o es un CRUD básico sin lógica de negocio relevante: el costo de mantener tres capas no se justifica.
- Se trata de componentes internos o de infraestructura sin comportamiento visible para el usuario (migraciones, configuración).
- El área de negocio no participa en la redacción ni en la revisión de los escenarios.
- Lo que se necesita verificar es rendimiento, seguridad o concurrencia.

5.2 Mutation Testing: cuándo aplicarlo y cuándo no

Conviene aplicarlo cuando:

- Se prueban reglas de negocio centrales y algoritmos críticos cuyo fallo afecta directamente la operación.
- Un módulo tiene alta cobertura de líneas y ramas, pero hay dudas sobre la solidez real de los `assert`.
- Se va a refactorizar una regla importante y se quiere una red de seguridad que detecte regresiones.

Conviene limitarlo o postergarlo cuando:

- El pipeline de CI/CD debe ser muy rápido en cada commit: conviene correrlo de forma periódica o solo sobre reglas modificadas.

- Las pruebas originales fallan o son inestables: primero hay que estabilizarlas, porque de lo contrario no se sabe si el fallo lo provocó el mutante.
- Las pruebas dependen de servicios remotos, esperas o datos compartidos: primero hay que aislar las reglas.
- El código es generado automáticamente, de terceros o sin comportamiento de negocio relevante.
- Los requisitos cambian continuamente: conviene acordar ejemplos antes de perseguir una puntuación.

5.3 Costos, rendimiento y mantenibilidad

Factor	BDD con Gherkin	Mutation Testing
Complejidad de adopción	Media: requiere aprender Gherkin, escribir step definitions y lograr participación del negocio.	Media: la herramienta es simple de ejecutar, pero interpretar sobrevivientes requiere criterio.
Costo humano	Medio-alto: acordar escenarios y programar sus step definitions.	Alto: inspeccionar sobrevivientes y descartar mutantes equivalentes.
Costo computacional	Bajo en lógica aislada (milisegundos); medio-lento si los escenarios atraviesan la API HTTP y la base de datos.	Proporcional a M × t (cantidad de mutantes por duración de la suite); la suite se ejecuta cientos o miles de veces.
Mantenibilidad alta si…	Los escenarios son declarativos y las step definitions están desacopladas y son reutilizables.	El análisis se acota a módulos críticos (configuración en `pyproject.toml`) y las pruebas tienen entradas y resultados explícitos.
Mantenibilidad baja si…	Los escenarios describen clics, campos o detalles cambiantes de la interfaz.	Se muta todo el código sin criterio o se persigue un umbral universal.
Momento de ejecución recomendado	En cada cambio, junto con el resto de las pruebas.	Periódicamente o sobre las reglas modificadas.

En la demo del grupo, la mutación de una sola clase generó 12 mutantes y se ejecutó a unas 11 mutaciones por segundo, lo que confirma que el costo es bajo cuando el análisis se acota a reglas puntuales.

5.4 Alternativas y complementos

Técnica	Relación con los temas investigados
TDD (Test-Driven Development)	Origen de BDD. Escribe primero la prueba unitaria, pero en lenguaje técnico; conviene cuando no hace falta involucrar al negocio.

Técnica	Relación con los temas investigados
ATDD / Specification by Example	Enfoques cercanos a BDD, centrados en criterios de aceptación acordados con ejemplos; BDD con Gherkin es una forma concreta de aplicarlos.
Pruebas unitarias parametrizadas (pytest)	Alternativa más liviana a BDD cuando las reglas son internas y no requieren lenguaje compartido.
Cobertura de líneas y ramas (coverage.py)	Métrica más barata que la mutación; sirve como primer filtro, pero no mide la fuerza de las aserciones.
Pruebas basadas en propiedades (Hypothesis)	Generan muchas entradas automáticamente; ayudan a descubrir casos borde y complementan la mutación.
Pruebas de contrato, rendimiento y concurrencia	Necesarias para lo que ninguna de las dos técnicas cubre: integración entre módulos, latencia y carreras.

5.5 Decisiones técnicas adoptadas

- **Python con pytest-bdd y mutmut:** ambas herramientas usan pytest, por lo que mutmut ejecuta directamente los escenarios BDD como suite de pruebas. Esto permite integrar las dos técnicas sin código adicional.
- **pytest-bdd en lugar de behave:** se integra con pytest, que es el ejecutor que mutmut necesita.
- **Docker:** mutmut requiere `fork()`, que no existe en Windows. El contenedor garantiza el mismo entorno Linux para todos los integrantes y para la demostración en clase, con versiones fijas de las dependencias.
- **Ejemplo independiente del TPI:** un cajero automático, con reglas simples y conocidas por todos, para que el foco esté en las técnicas y no en el dominio.
- **Mutación acotada a `src/`:** solo se muta el código de producción; las pruebas permanecen intactas durante el análisis.

6. Implementación práctica integrada

6.1 Objetivo y escenario elegido

La demo combina ambas técnicas en un mismo escenario: el retiro de dinero en un cajero automático. BDD define y verifica el comportamiento esperado; Mutation Testing evalúa si esos escenarios detectan errores; y los casos no cubiertos que se identifican se incorporan como nuevos escenarios en Gherkin.

Se eligió un dominio con reglas simples y conocidas por todos, para que el foco esté en la interacción entre las técnicas. La clase “Cuenta Bancaria” aplica tres reglas:

1. Si el monto a retirar es menor o igual a 0 → `MONTO_INVALIDO`.
2. Si el monto supera el saldo disponible → `SALDO_INSUFFICIENTE` (el saldo no cambia).

3. En otro caso, se descuenta el monto → `RETIRO_EXITOSO`.

6.2 Estructura del proyecto

```
demo-bdd-mutation-testing/
├── src/
│   └── cuenta.py              # Lógica de negocio (código que muta mutmut)
├── tests/
│   ├── features/
│   │   └── retiro.feature     # Escenarios BDD en Gherkin (español)
│   └── test_retiro.py         # Step definitions con pytest-bdd
├── pyproject.toml             # Configuración de pytest y mutmut
├── requirements.txt           # Versiones fijas de las dependencias
├── Dockerfile                 # Imagen del contenedor
├── docker-compose.yml         # Levanta el contenedor con un comando
├── .gitignore / .dockerignore # Excluyen mutants/, entornos y cachés
├── GUIA.md                    # Guía de instalación
└── PASOS_DEMO.txt             # Comandos paso a paso de la demo
```

6.3 Código, escenarios y step definitions

Código de producción (`src/cuenta.py`):

```
class CuentaBancaria:
    def __init__(self, saldo_inicial: float):
        self.saldo = saldo_inicial

    def retirar(self, monto: float) -> str:
        if monto <= 0:
            return "MONTO_INVALIDO"
        # Regla de negocio: si el monto supera el saldo disponible, rechaza
        if monto > self.saldo:
            return "SALDO_INSUFFICIENTE"

        self.saldo -= monto
        return "RETIRO_EXITOSO"
```

Escenarios iniciales (`tests/features/retiro.feature`):

```
# language: es
Característica: Retiro de dinero en Cajero Automatico

  Escenario: Retiro exitoso cuando hay saldo suficiente
    Dado que el cliente tiene un saldo disponible de 1000 pesos
    Cuando solicita retirar 300 pesos
    Entonces la operacion es "RETIRO_EXITOSO"
    Y el nuevo saldo disponible es de 700 pesos

  Escenario: Rechazo de retiro por saldo insuficiente
    Dado que el cliente tiene un saldo disponible de 500 pesos
    Cuando solicita retirar 600 pesos
    Entonces la operacion es "SALDO_INSUFFICIENTE"
```

```
    Y el nuevo saldo disponible es de 500 pesos

  @limite
  Escenario: Retiro por el monto exacto del saldo disponible
    Dado que el cliente tiene un saldo disponible de 500 pesos
    Cuando solicita retirar 500 pesos
    Entonces la operacion es "RETIRO_EXITOSO"
    Y el nuevo saldo disponible es de 0 pesos
```

Step definitions (`tests/test_retiro.py`):conectan cada frase del Gherkin con una función de Python. Los pasos *Dado* y *Cuando* invocan la clase real CuentaBancaria, y los pasos *Entonces* verifican el resultado con assert. Como las pruebas ejecutan el código real, mutmut puede detectar los cambios que introduce.

```
from pytest_bdd import scenarios, given, when, then, parsers
from cuenta import CuentaBancaria

scenarios("features/retiro.feature")

@given(parsers.parse("que el cliente tiene un saldo disponible de {saldo:d} pesos"),
target_fixture="cuenta")
def inicializar_cuenta(saldo):
    return CuentaBancaria(saldo_inicial=float(saldo))

@when(parsers.parse("solicita retirar {monto:d} pesos"), target_fixture="resultado")
def realizar_retiro(cuenta, monto):
    return cuenta.retirar(monto=float(monto))

@then(parsers.parse('la operacion es "{esperado}"'))
def verificar_resultado(resultado, esperado):
    assert resultado == esperado

@then(parsers.parse("el nuevo saldo disponible es de {saldo_esperado:d} pesos"))
def verificar_saldo(cuenta, saldo_esperado):
    assert cuenta.saldo == float(saldo_esperado)
```

Configuración (`pyproject.toml`): indica a pytest dónde están el código y las pruebas, y a mutmut qué código mutar (src/) y qué pruebas usar para detectar los mutantes (tests/, es decir, los propios escenarios BDD).

```
[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
markers = ["limite: casos de borde"]

[tool.mutmut]
source_paths = ["src/"]
pytest_add_cli_args_test_selection = ["tests/"]
```

6.4 Contenedor Docker

La herramienta mutmut necesita la función del sistema operativo fork(), que permite crear copias de un proceso en ejecución y que solo existe en sistemas tipo Unix, como Linux. Como en Windows no está disponible, la demo se ejecuta dentro de un contenedor Docker con Linux. Además, el contenedor garantiza que todos los integrantes trabajen con el mismo entorno y las mismas versiones de las dependencias, fijadas en requirements.txt (pytest 9.2.0, pytest-bdd 9.0.1 y mutmut 3.8.1).

El archivo Dockerfile define cómo se construye la imagen del contenedor: parte de una imagen oficial de Python 3.13 sobre Linux, instala las dependencias y copia el proyecto.

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["bash"]
```

El archivo docker-compose.yml permite levantar el contenedor con un solo comando (docker compose run --rm demo) y abre una terminal interactiva dentro de él.

```
services:
  demo:
    build: .
    volumes:
      - .:/app
    stdin_open: true
    tty: true
```

El volumen .:/app monta la carpeta del proyecto dentro del contenedor, de modo que los cambios en el .feature se reflejan al instante sin reconstruir la imagen. La opción --rm del comando elimina el contenedor al salir, sin afectar los archivos del proyecto.

6.5 Ejecución y evidencias

La demostración se ejecuta dentro del contenedor Docker, por lo que requiere tener Docker Desktop en ejecución. A continuación se detalla cada comando en el orden en que se utiliza, qué hace y cuál es el resultado esperado.

Paso 1. Levantar el contenedor

```
docker compose run --rm demo
```

Qué hace: Construye la imagen definida en el `Dockerfile` (solo la primera vez) e inicia un contenedor Linux con Python, pytest, pytest-bdd y mutmut ya instalados. Al terminar, abre una terminal dentro del contenedor, donde se ejecutan todos los comandos siguientes.

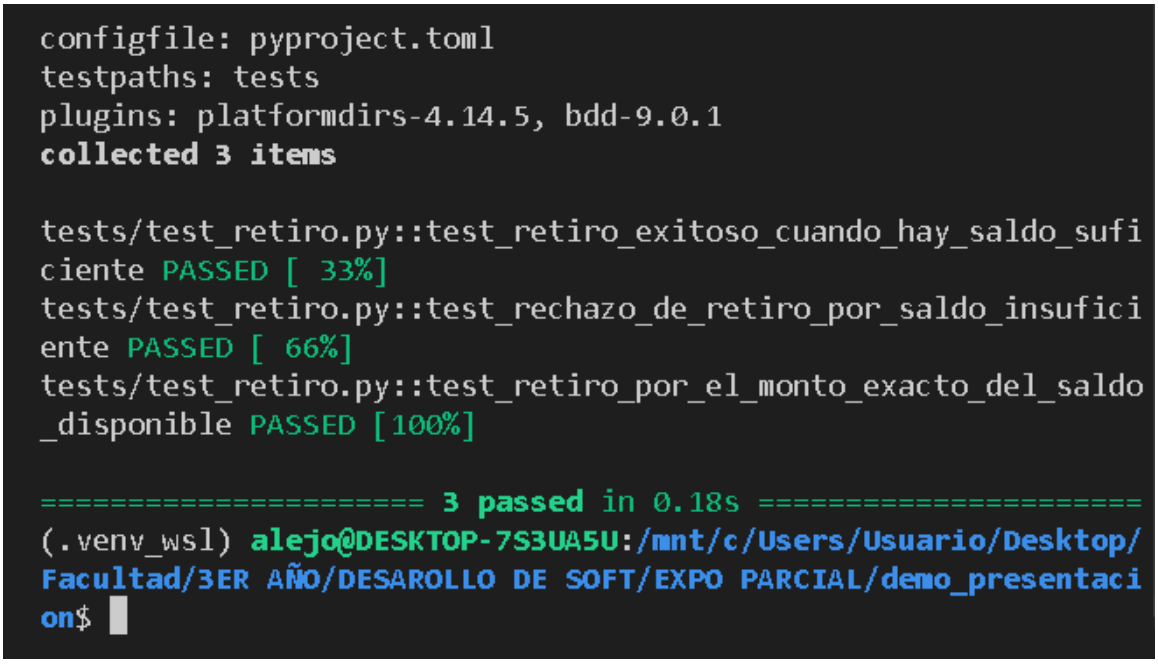
Resultado esperado: Se abre la terminal del contenedor, con un indicador del estilo `root@<id>:/app#`.

Paso 2. Ejecutar las pruebas BDD

```
pytest -v
```

Qué hace: pytest-bdd lee el archivo `retiro.feature`, convierte cada escenario escrito en Gherkin en una prueba y la ejecuta a través de las step definitions, que invocan el código real de `CuentaBancaria`. La opción `-v` muestra el resultado de cada escenario por separado.

Resultado esperado: Los 3 escenarios se aprueban (`3 passed`). Este resultado es la línea base: todas las pruebas pasan sobre el código original.



Paso 3. Borrar resultados anteriores de mutmut

```
rm -rf mutants
```

Qué hace: Elimina la carpeta `mutants`, donde mutmut guarda los mutantes y los resultados de ejecuciones anteriores. Así el análisis siguiente parte de cero.

Resultado esperado: El comando no muestra ninguna salida.

Paso 4. Ejecutar mutation testing

```
mutmut run
```

Qué hace: mutmut verifica que las pruebas pasen sobre el código original, genera los mutantes de `src/cuenta.py` y ejecuta los escenarios BDD contra cada uno de ellos, registrando si fueron eliminados o sobrevivieron.

Resultado esperado: Se evalúan 12 mutantes: 8 eliminados y 4 sobrevivientes. La puntuación de mutación es 8 / (8 + 4) × 100 = 66,7 %. Aunque las pruebas BDD pasaban en verde, hay errores que no detectan.

```
Facultad/3ER AÑO/DESAROLLO DE SOFT/EXPO PARCIAL/demo_presentaci
on$ mutmut run
⠇ Generating mutants
    done in 394ms (1 files mutated, 0 ignored, 0 unmodified)
⠏ Running stats
    done
⠇ Running clean tests
    done
⠇ Running forced fail test
    done
Running mutation testing
⠇ 12/12  🎉 8 🤔 0  ⏰ 0  🤔 0  😕 4  🔇 0  🧙 0
10.73 mutations/second
(.venv_wsl) alejo@DESKTOP-7S3UA5U:/mnt/c/Users/Usuario/Desktop/
Facultad/3ER AÑO/DESAROLLO DE SOFT/EXPO PARCIAL/demo_presentaci
on$
```

Paso 5. Listar los mutantes sobrevivientes

```
mutmut results
```

Qué hace: Muestra la lista de mutantes que no fueron eliminados, con su nombre y su estado.

Resultado esperado: Aparecen cuatro mutantes del método `retirar` (`retirar__mutmut_1` a `retirar__mutmut_4`) con el estado `survived`.

```
10.73 mutations/second
(.venv_wsl) alejo@DESKTOP-7S3UA5U:/mnt/c/Users/Usuario/Desktop/
Facultad/3ER AÑO/DESAROLLO DE SOFT/EXPO PARCIAL/demo_presentaci
on$ mutmut results
    cuenta.x‖CuentaBancaria‖retirar__mutmut_1: survived
    cuenta.x‖CuentaBancaria‖retirar__mutmut_2: survived
    cuenta.x‖CuentaBancaria‖retirar__mutmut_3: survived
    cuenta.x‖CuentaBancaria‖retirar__mutmut_4: survived
(.venv_wsl) alejo@DESKTOP-7S3UA5U:/mnt/c/Users/Usuario/Desktop/
Facultad/3ER AÑO/DESAROLLO DE SOFT/EXPO PARCIAL/demo_presentaci
on$
```

Paso 6. Ver el cambio introducido por un mutante

```
mutmut show 'cuenta.x‖CuentaBancaria‖retirar__mutmut_1'
```

Qué hace: Muestra la diferencia entre el código original y el código mutado: la línea original aparece marcada con `-` y la línea modificada con `+`. El nombre del mutante se copia de la salida de `mutmut results`.

Resultado esperado: Para el mutante 1 se observa que la condición `monto <= 0` fue reemplazada por `monto < 0`.

Análisis de los sobrevivientes. Los cuatro están en la regla del monto inválido, que ningún escenario ejercitaba:

Mutante	Cambio introducido	Por qué sobrevivió
1	`monto <= 0` → `monto < 0`	Ningún escenario retira 0 pesos.
2	`monto <= 0` → `monto <= 1`	Ningún escenario retira 1 peso.
3	`"MONTO_INVALIDO"` → `"XXMONTO_INVALIDOXX"`	Ningún escenario verifica el mensaje de monto inválido.
4	`"MONTO_INVALIDO"` → `"monto_invalido"`	Ídem.

Los otros ocho mutantes fueron eliminados: por ejemplo, `monto > saldo` → `monto >= saldo` lo detecta el escenario `@limite` del monto exacto, y `-=` → `+=` lo detecta la verificación del nuevo saldo.

Paso 7. Agregar los escenarios faltantes en Gherkin

Qué se hace: Se abre el archivo `tests/features/retiro.feature` y se agregan al final dos escenarios nuevos, sin modificar el código de producción:

```
  Escenario: Rechazo de retiro con monto cero
    Dado que el cliente tiene un saldo disponible de 500 pesos
    Cuando solicita retirar 0 pesos
    Entonces la operacion es "MONTO_INVALIDO"
    Y el nuevo saldo disponible es de 500 pesos

  Escenario: Retiro del monto mínimo
    Dado que el cliente tiene un saldo disponible de 500 pesos
    Cuando solicita retirar 1 pesos
    Entonces la operacion es "RETIRO_EXITOSO"
    Y el nuevo saldo disponible es de 499 pesos
```

El escenario de monto cero apunta a los mutantes 1, 3 y 4, y el de monto mínimo, al mutante 2. Gracias al volumen montado en `docker-compose.yml`, el contenedor ve el cambio al instante, sin necesidad de reiniciarlo.

Paso 8. Volver a ejecutar las pruebas BDD

```
pytest -v
```

Qué hace: Ejecuta nuevamente los escenarios del archivo `.feature`, ahora incluyendo los dos agregados.

Resultado esperado: Los 5 escenarios se aprueban (`5 passed`).

```
collected 5 items

tests/test_retiro.py::test_retiro_exitoso_cuando_hay_saldo_sufi
ciente PASSED [ 20%]
tests/test_retiro.py::test_rechazo_de_retiro_por_saldo_insufici
ente PASSED [ 40%]
tests/test_retiro.py::test_retiro_por_el_monto_exacto_del_saldo
_disponible PASSED [ 60%]
tests/test_retiro.py::test_rechazo_de_retiro_con_monto_cero PAS
SED [ 80%]
tests/test_retiro.py::test_retiro_del_monto_mínimo PASSED [100%
]

====================== 5 passed in 0.26s ======================
(.venv_wsl) alejo@DESKTOP-7S3UA5U:/mnt/c/Users/Usuario/Desktop/
Facultad/3ER AÑO/DESAROLLO DE SOFT/EXPO PARCIAL/demo_presentaci
```

Paso 9. Volver a ejecutar mutation testing

```
rm -rf mutants
mutmut run
```

Qué hace: Primero se borra la carpeta `mutants`, porque mutmut reutiliza los resultados anteriores cuando el código fuente no cambió, y en este caso solo se modificaron las pruebas. Luego se repite el análisis completo.

Resultado esperado: Los 12 mutantes son eliminados y no queda ningún sobreviviente. La puntuación de mutación pasa a 12 / (12 + 0) × 100 = 100 %.

```
Facultad/3ER AÑO/DESAROLLO DE SOFT/EXPO PARCIAL/demo_presentaci
on$ mutmut run
⠋ Generating mutants
    done in 377ms (1 files mutated, 0 ignored, 0 unmodified)
⠙ Running stats
    done
⠹ Running clean tests
    done
⠸ Running forced fail test
    done
Running mutation testing
⠼ 12/12  🎉 12 🫥 0  ⏰ 0  🤔 0  😐 0  🙊 0  🧙 0
11.45 mutations/second
(.venv_wsl) alejo@DESKTOP-7S3UA5U:/mnt/c/Users/Usuario/Desktop/
Facultad/3ER AÑO/DESAROLLO DE SOFT/EXPO PARCIAL/demo_presentaci
on$
```

Paso 10. Salir del contenedor

```
exit
```

Qué hace: Cierra la terminal del contenedor. Por la opción `--rm`, el contenedor se elimina automáticamente, sin afectar los archivos del proyecto.

Resultado esperado: Se vuelve a la terminal de la computadora. Para repetir la demostración desde el estado inicial, se ejecuta fuera del contenedor `git checkout tests/features/retiro.feature`, que deja el archivo `.feature` con sus tres escenarios originales.

6.6 Aporte de la integración

- **BDD sola** mostró todas las pruebas en verde (3 de 3), lo que daba una falsa sensación de seguridad.
- **Mutation Testing** demostró que una regla completa (monto inválido) no estaba verificada, y lo cuantificó: 66,7 %.
- **La corrección se hizo en Gherkin**, de modo que el caso faltante quedó documentado en lenguaje de negocio y la puntuación subió a 100 %.
- **mutmut usó los escenarios BDD como suite de pruebas**, sin código adicional, porque ambas herramientas comparten pytest.

El ciclo resultante es: especificar con BDD → evaluar con mutación → completar la especificación con BDD.

7. Conclusiones

BDD con Gherkin y Mutation Testing no compiten: se complementan. BDD define **qué** debe hacer el sistema, en un lenguaje que entiende todo el equipo; Mutation Testing comprueba que esos escenarios **realmente** lo controlen.

Tener en cuenta:

- Que todas las pruebas pasen no garantiza que sean buenas. En la demo, 3 de 3 escenarios aprobados convivían con una regla completa sin verificar, que Mutation Testing expuso con una puntuación de 66,7 %.
- La mejora se expresó en Gherkin, por lo que el caso faltante quedó documentado en lenguaje de negocio y la puntuación llegó al 100 %.
- Ambas técnicas tienen costos: BDD exige mantener tres capas y la participación del negocio; Mutation Testing exige tiempo de cómputo y análisis humano. Por eso conviene aplicarlas a reglas de negocio críticas, como las de vencimiento, destinatario y asignación única de ofertas en el módulo M5 del TPI.
- Ninguna reemplaza las pruebas de contrato, concurrencia, rendimiento o seguridad, que la plataforma distribuida también necesita.
- El uso de Docker y de un repositorio Git hizo que la demo sea reproducible por cualquier integrante con un solo comando.

8. Repositorio de código

La implementación completa está publicada en: **github.com/Alejo13007/demo-bdd-mutation-testing**

El repositorio se publicó en su estado inicial (3 escenarios, 8 mutantes eliminados y 4 sobrevivientes) para poder reproducir la demostración completa. El archivo `PASOS_DEMO.txt` detalla todos los comandos y la corrección.

Requisitos: Docker Desktop en ejecución y Git.

9. Fuentes

BDD con Gherkin

1. North, D. (2006). *Introducing BDD*. Origen del término y motivación de BDD respecto de TDD. dannorth.net/introducing-bdd
2. Cucumber. *Gherkin Reference*. Palabras clave, estructura de pasos, Background y Scenario Outline. cucumber.io/docs/gherkin/reference
3. Cucumber. *Writing better Gherkin*. Estilo declarativo frente a imperativo y escenarios como documentación viva. cucumber.io/docs/bdd/better-gherkin
4. pytest-bdd. Documentación oficial. Instalación, scenarios, step definitions, parsers y reportes. pytest-bdd.readthedocs.io
5. behave. Documentación oficial. Alternativa de BDD para Python con ejecutor propio. behave.readthedocs.io
6. ITDO. *Ejemplos BDD (Behavior Driven Development) con Gherkin*. itdo.com/blog/ejemplos-bdd-behavior-driven-development-con-gherkin

Mutation Testing

1. Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y. y Harman, M. (2019). *Mutation Testing Advances: An Analysis and Survey*. Advances in Computers, 112, 275–378. DOI: 10.1016/bs.adcom.2018.03.015. discovery.ucl.ac.uk/id/eprint/10056704
2. PIT. *Mutation Testing*. Fundamentos de la técnica y diferencias respecto de la cobertura tradicional. pitest.org
3. PIT. *Basic Concepts*. Operadores, mutantes equivalentes y estados del análisis. pitest.org/quickstart/basic_concepts
4. Stryker Mutator. *Mutant states and metrics*. stryker-mutator.io/docs/mutation-testing-elements/mutant-states-and-metrics
5. mutmut. Documentación oficial. Instalación, requisitos, configuración y análisis de resultados. mutmut.readthedocs.io

Herramientas comunes

1. pytest. *How to parametrize fixtures and test functions*. docs.pytest.org/en/stable/how-to/parametrize.html
2. pytest. *How to mark test functions with attributes*. docs.pytest.org/en/stable/how-to/mark.html
3. Docker. Documentación oficial de Dockerfile y Docker Compose. docs.docker.com